

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

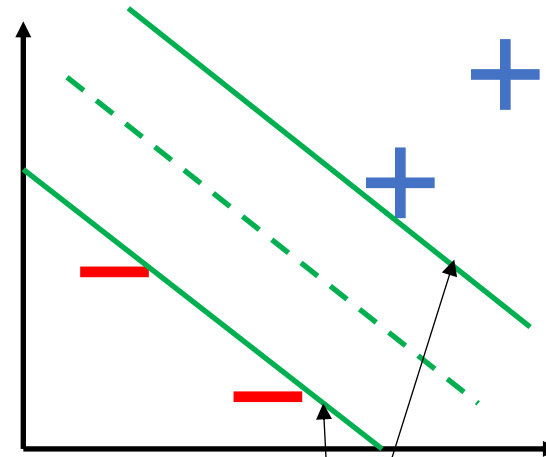
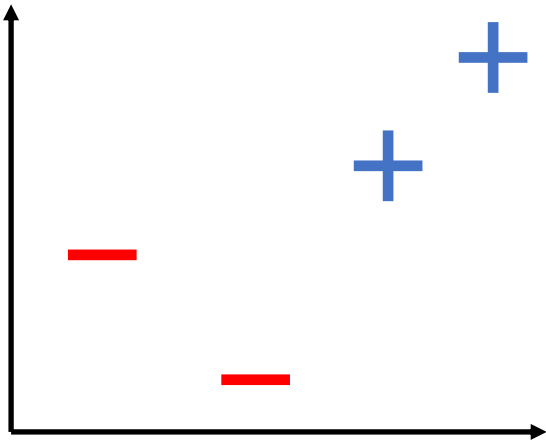
파이널 프로젝트

AI/BigData 인텐시브 과정

Section 7. 서포트 벡터 머신

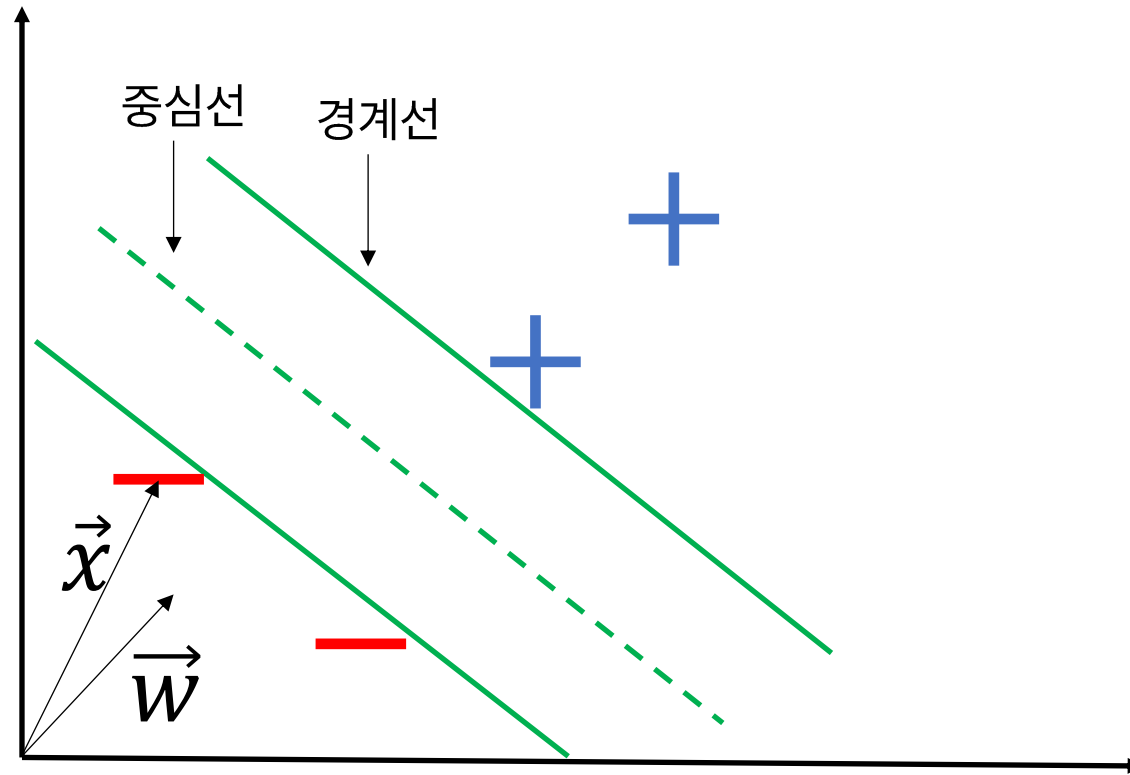
Section 7-1. 서포트 벡터 머신 개념

서포트 벡터 머신



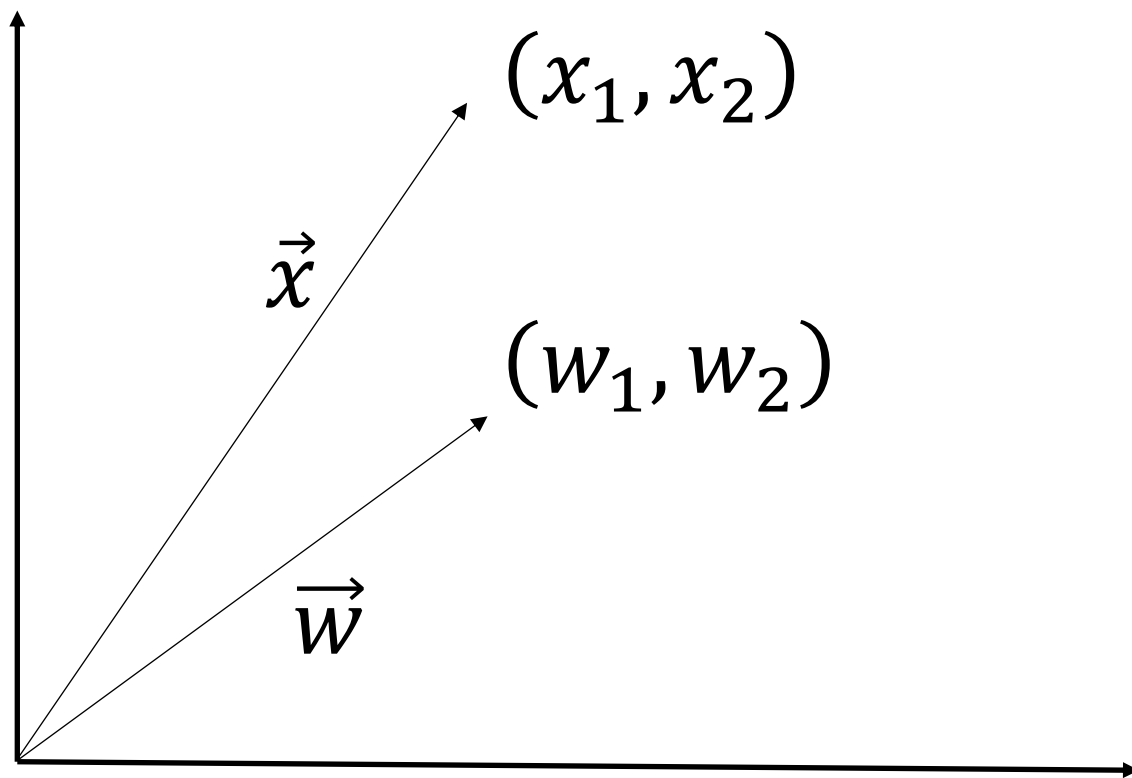
서포트 벡터

중심선과 경계선

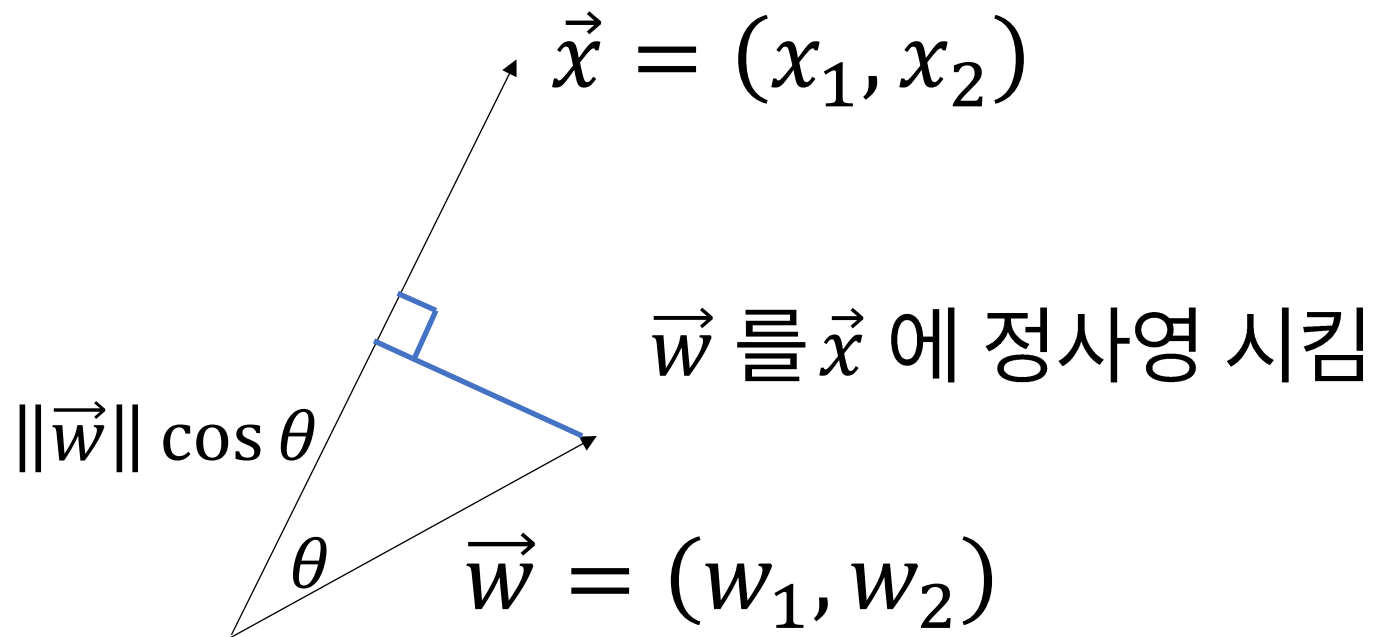


$$\vec{w} = (w_1, w_2) \quad - \text{중심선에 수직인 벡터}$$

벡터 표현식

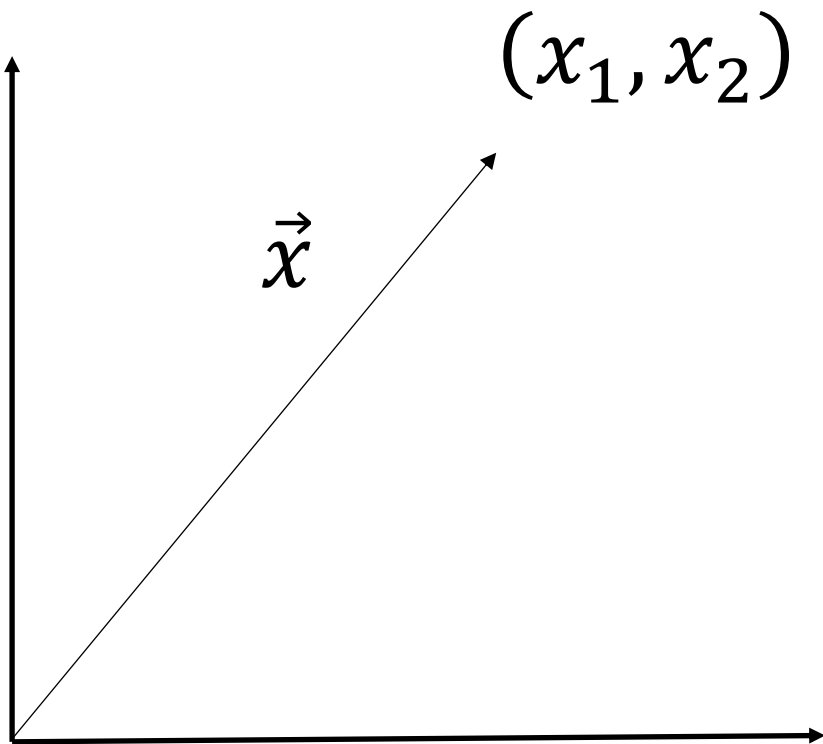


내적, 정사영



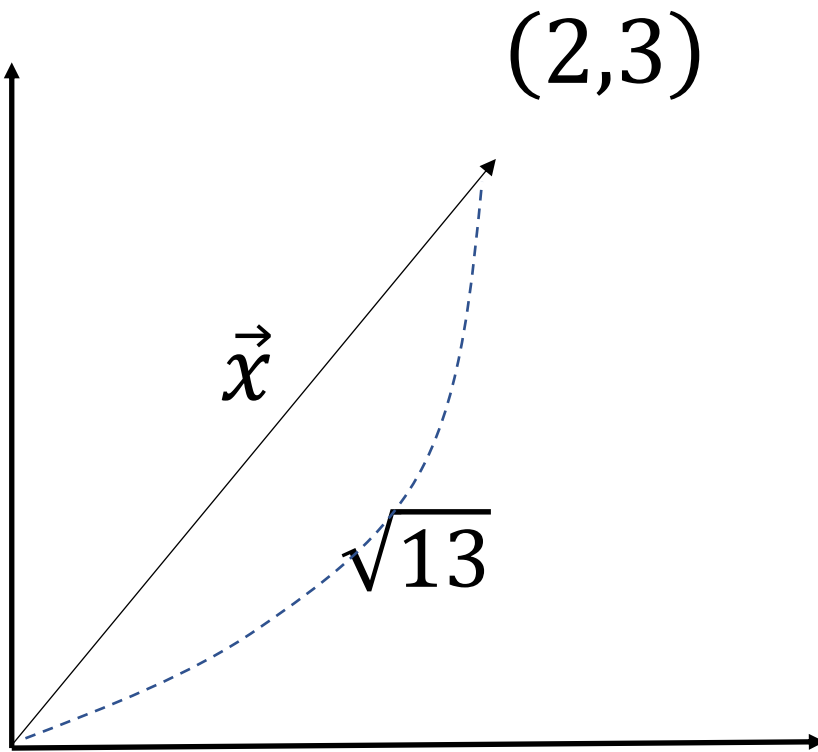
$$\vec{w} \cdot \vec{x} = \|\vec{w}\| \|\vec{x}\| \cos \theta$$

벡터의 길이



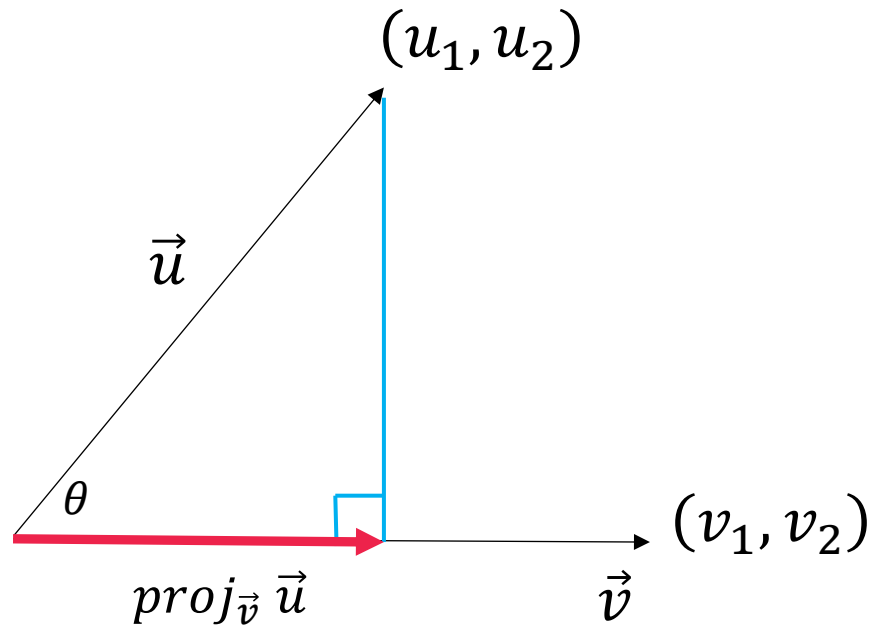
$$\|\vec{x}\| = \sqrt{x_1^2 + x_2^2}$$

벡터의 길이



$$\begin{aligned}\|\vec{x}\| &= \sqrt{x_1^2 + x_2^2} \\ &= \sqrt{2^2 + 3^2} \\ &= \sqrt{4 + 9} \\ &= \sqrt{13}\end{aligned}$$

정사영(projection)

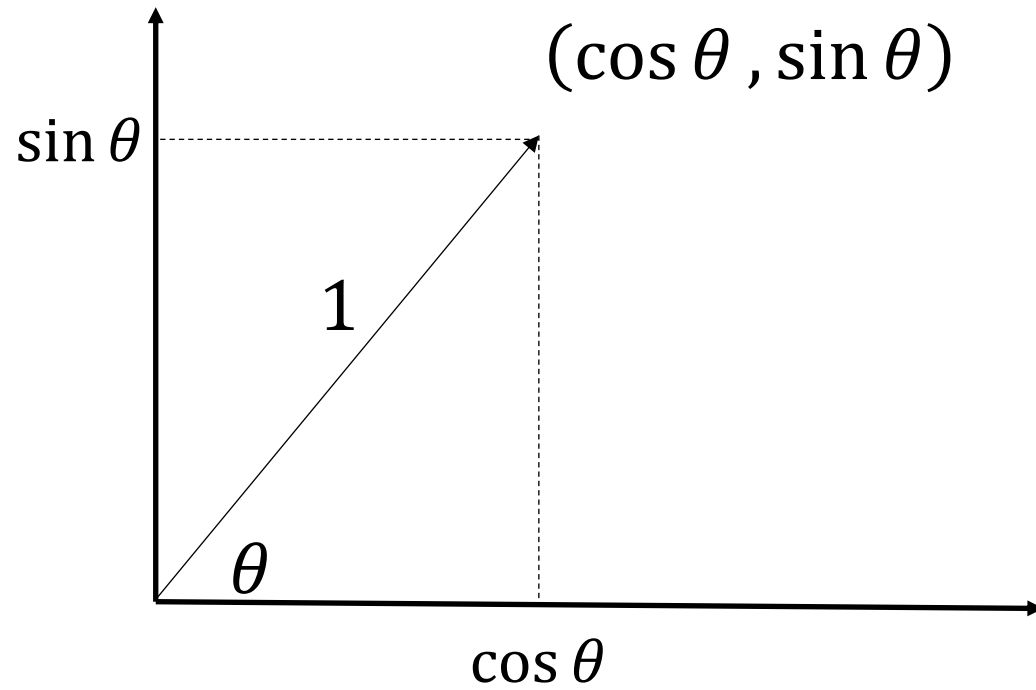


$\vec{x}$ 를 $\vec{y}$ 에 정사영한 벡터의 길이

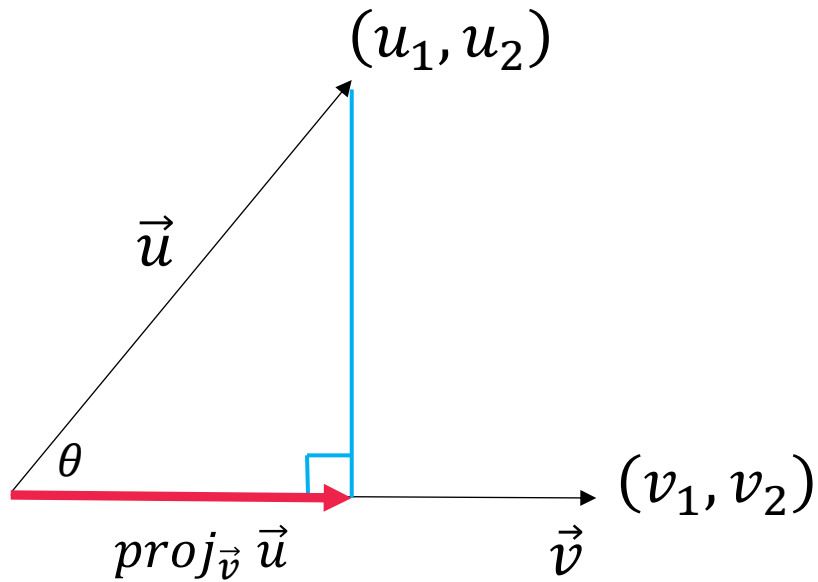
$$\|proj_{\vec{v}} \vec{u}\| = \|\vec{u}\| \cos \theta$$

?

삼각함수와 좌표평면



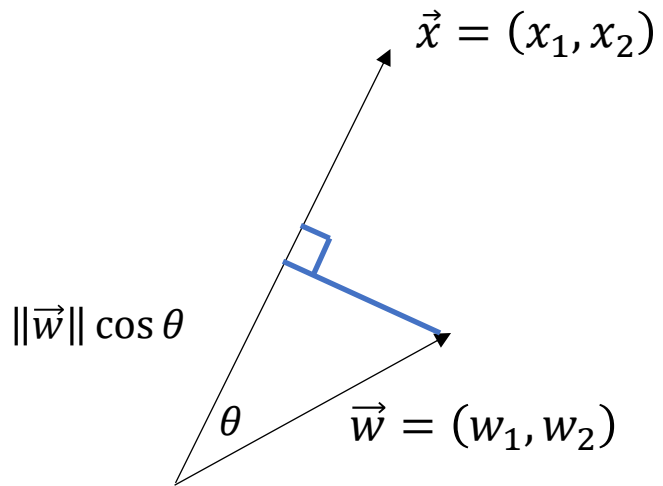
삼각함수와 좌표평면



$\vec{x}$ 를 $\vec{y}$ 에 정사영한 벡터의 길이

$$\|proj_{\vec{v}} \vec{u}\| = \|\vec{u}\| \cos \theta$$

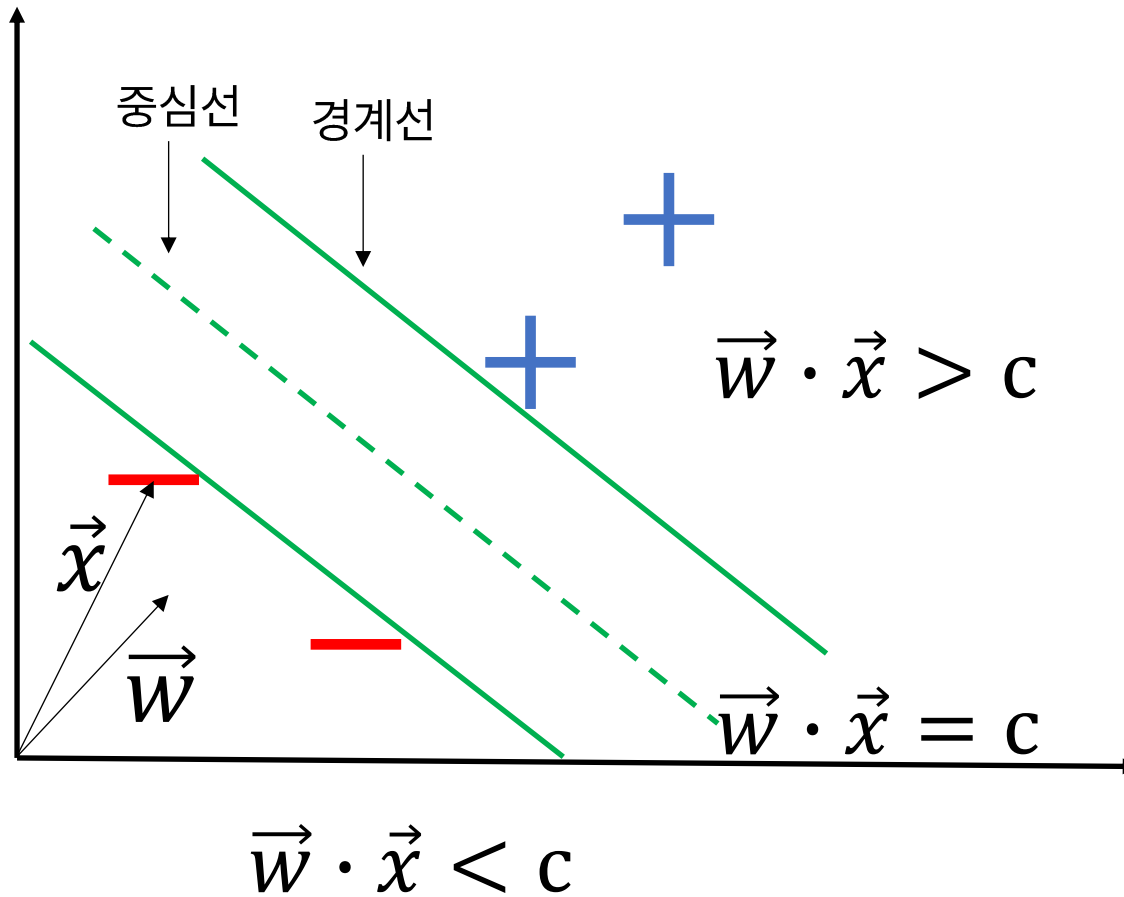
내적



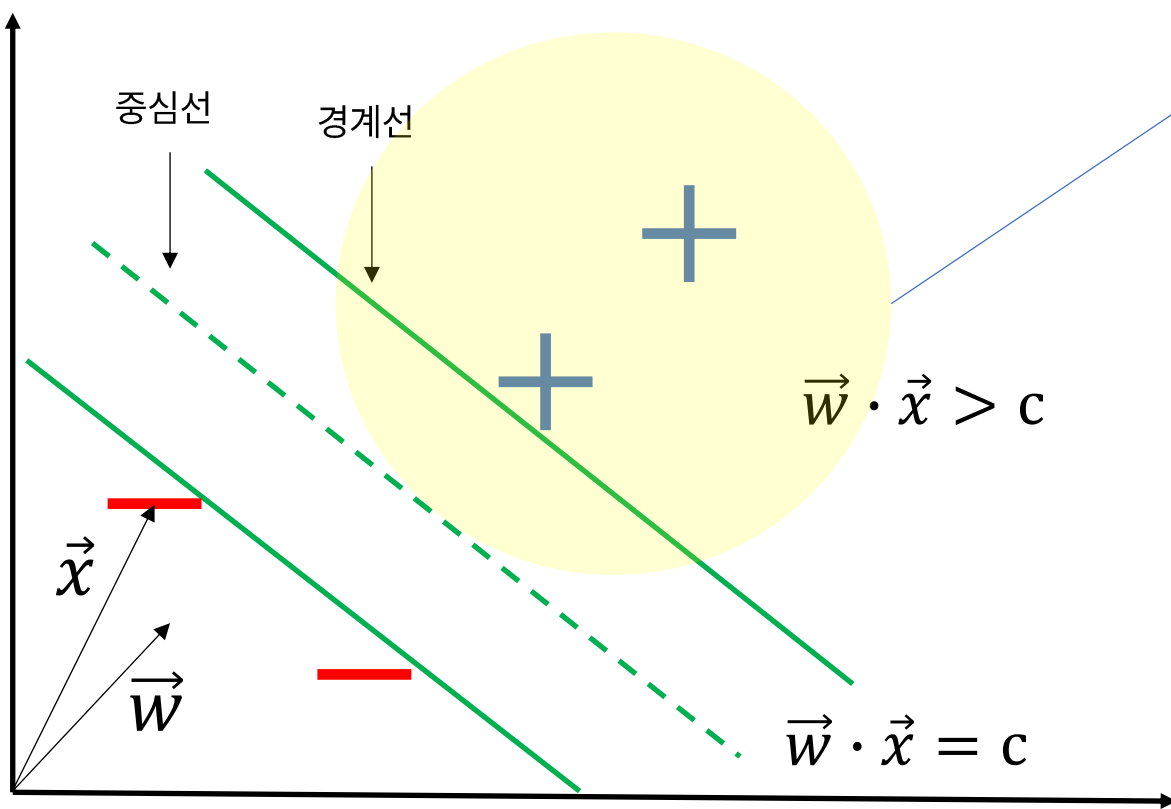
$$\vec{w} \cdot \vec{x} = \|\vec{w}\| \|\vec{x}\| \cos \theta$$

$$\vec{w} \cdot \vec{x} = w_1 x_1 + w_2 x_2$$

내적값으로 판별하기



결정 조건



+ 영역에 속하려면

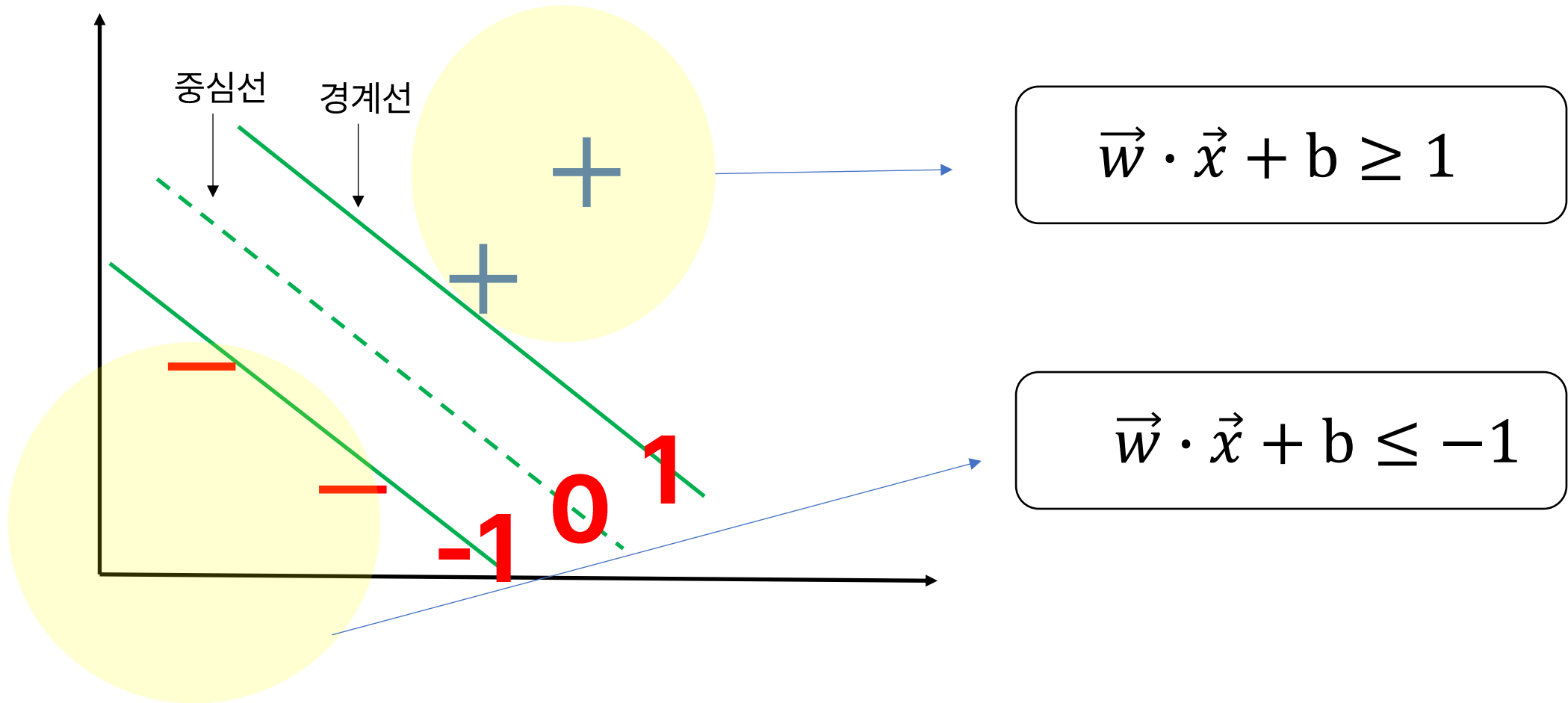
$$\vec{w} \cdot \vec{x} > c$$



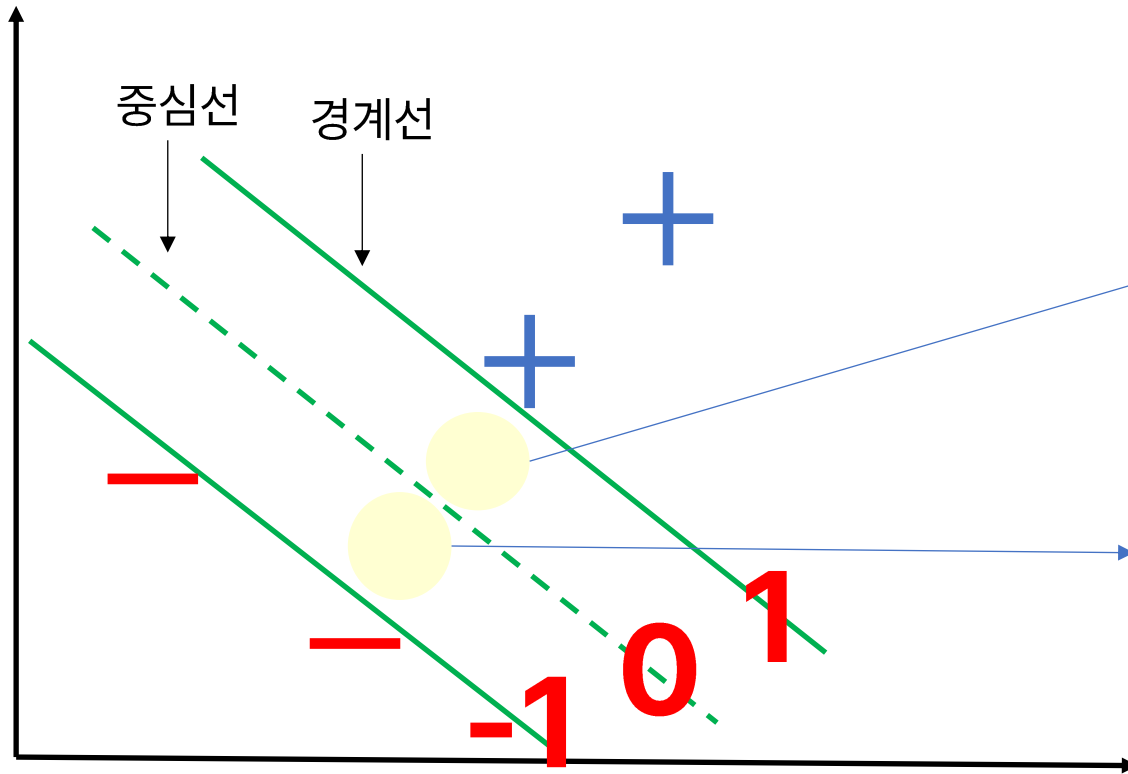
$$\vec{w} \cdot \vec{x} + b > 0$$

Decision Rule

서포트 벡터 머신



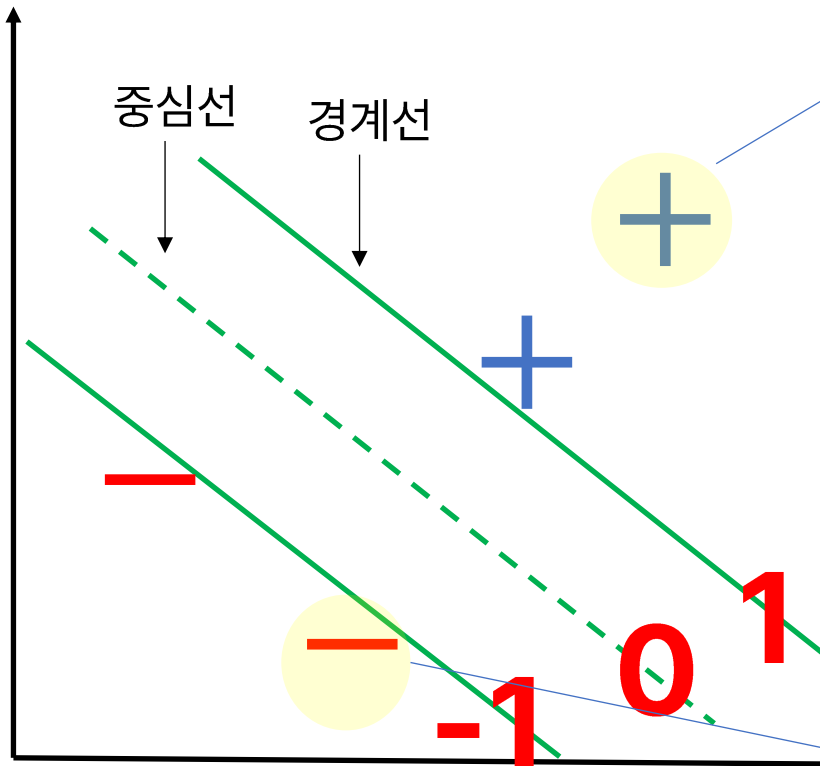
서포트 벡터 머신



$$0 \leq \vec{w} \cdot \vec{x} + b \leq 1$$

$$-1 \leq \vec{w} \cdot \vec{x} + b \leq 0$$

서포트 벡터 머신



+영역에 있는 x 벡터: $\vec{x}_+$

$$\vec{w} \cdot \vec{x}_+ + b \geq 1$$

$$\vec{w} \cdot \vec{x}_- + b \leq -1$$

-영역에 있는 x 벡터: $\vec{x}_-$

제약식 두 개를 하나로 만들기

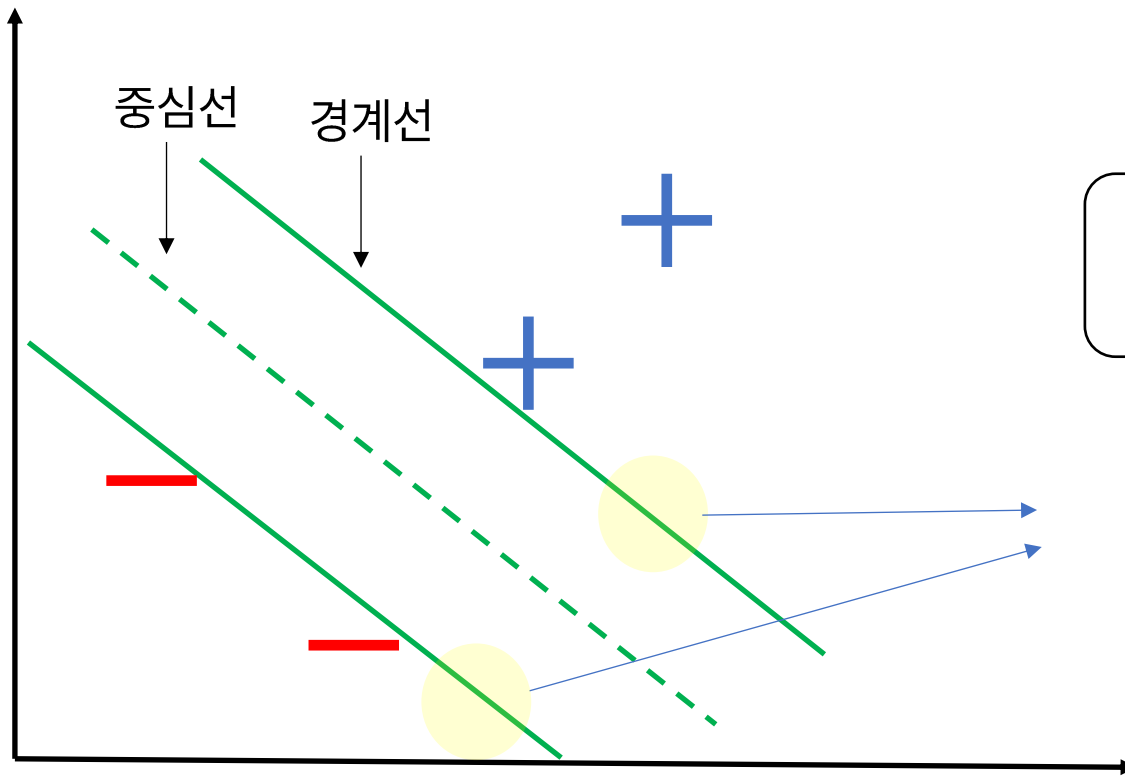
$$y_i = \begin{cases} 1, & x \text{가 } + \text{ 영역에 있을 때} \\ -1, & x \text{가 } - \text{ 영역에 있을 때} \end{cases}$$

$$\vec{w} \cdot \vec{x}_+ + b \geq 1 \quad \longrightarrow \quad y_i(\vec{w} \cdot \vec{x}_+ + b) \geq 1$$

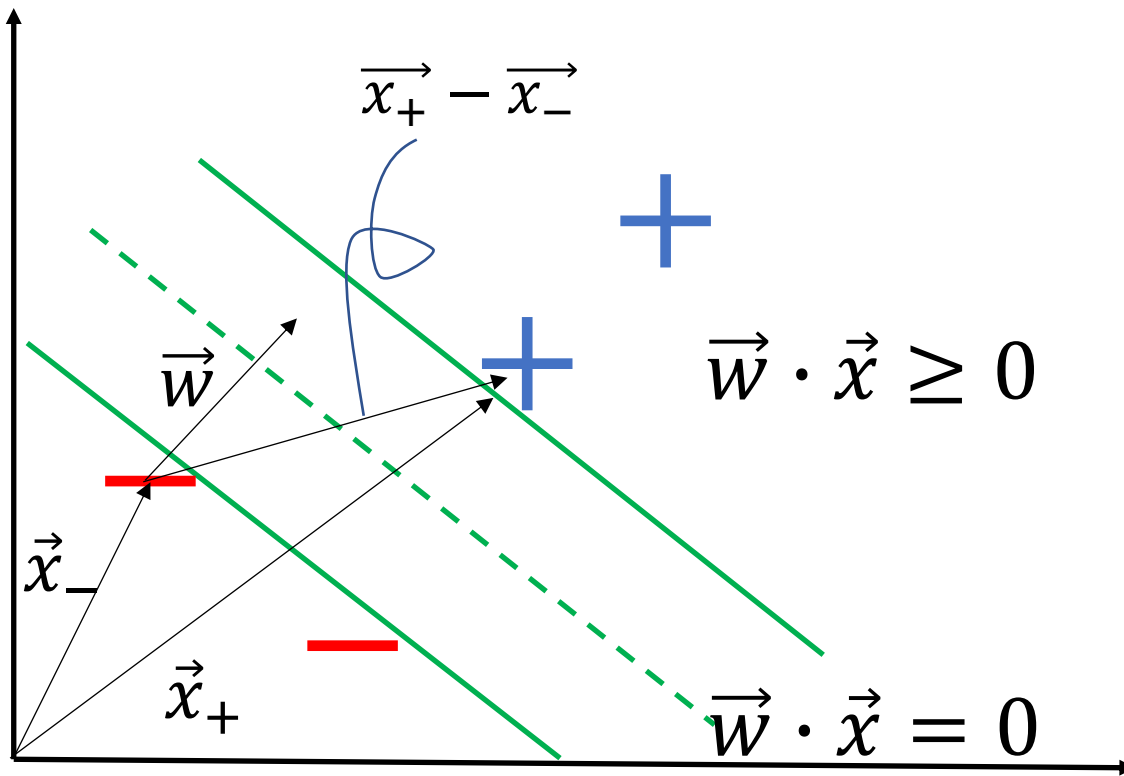
$$\vec{w} \cdot \vec{x}_- + b \leq -1 \quad \longrightarrow \quad y_i(\vec{w} \cdot \vec{x}_- + b) \geq 1$$

$$y_i(\vec{w} \cdot \vec{x} + b) - 1 \geq 0$$

경계선에 걸쳐 있는 데이터 표현



서포트 벡터간 너비(width)



너비(width)

$$(\vec{x}_+ - \vec{x}_-) \cdot \frac{\vec{w}}{\|\vec{w}\|}$$

$$\vec{w} \cdot \vec{x} \geq 0$$

$$\vec{w} \cdot \vec{x} = 0$$

너비 간단하게 표현하기

$$\text{제약식} \quad y_i(\vec{w} \cdot \vec{x}_i + b) - 1 = 0$$

i) + 경계선

$$y_i(\vec{w} \cdot \vec{x}_i + b) - 1 = 0$$

$$\Leftrightarrow 1 \times (\vec{w} \cdot \vec{x}_+ + b) - 1 = 0$$

$$\Leftrightarrow \vec{w} \cdot \vec{x}_+ + b - 1 = 0$$

$$\Leftrightarrow \vec{w} \cdot \vec{x}_+ = 1 - b$$

너비 간단하게 표현하기

$$\text{제약식} \quad y_i(\vec{w} \cdot \vec{x}_i + b) - 1 = 0$$

ii) - 경계선

$$y_i(\vec{w} \cdot \vec{x}_i + b) - 1 = 0$$

$$\Leftrightarrow -1 \times (\vec{w} \cdot \vec{x}_- + b) - 1 = 0$$

$$\Leftrightarrow -\vec{w} \cdot \vec{x}_- - b - 1 = 0$$

$$\Leftrightarrow -\vec{w} \cdot \vec{x}_- = 1 + b$$

너비 간단하게 표현하기

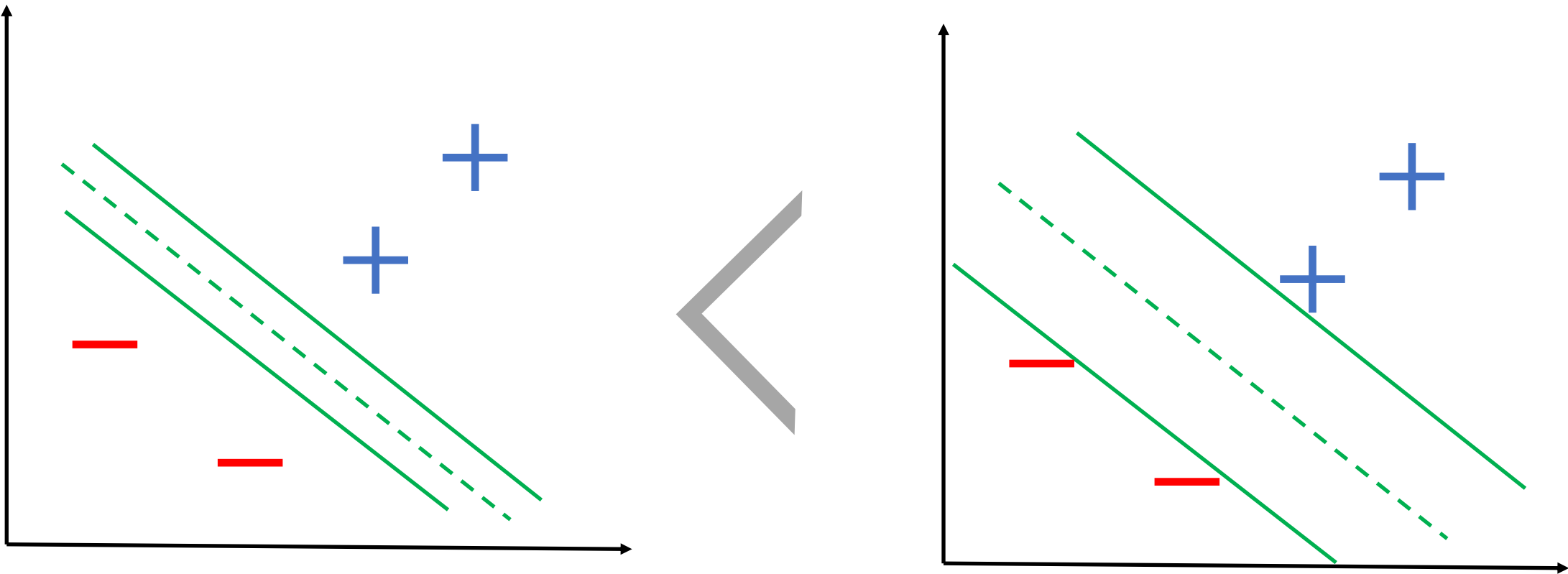
i), ii)를 이용

$$\begin{aligned}(\overrightarrow{x_+} - \overrightarrow{x_-}) \cdot \frac{\overrightarrow{w}}{\|\overrightarrow{w}\|} &= \frac{\overrightarrow{w} \cdot \overrightarrow{x_+} - \overrightarrow{w} \cdot \overrightarrow{x_-}}{\|\overrightarrow{w}\|} \\ &= \frac{1 - b + 1 + b}{\|\overrightarrow{w}\|} \\ &= \frac{2}{\|\overrightarrow{w}\|}\end{aligned}$$

$$-\overrightarrow{w} \cdot \overrightarrow{x_-} = 1 + b$$

$$\overrightarrow{w} \cdot \overrightarrow{x_+} = 1 - b$$

너비 최대화



서포트 벡터 간 너비가 클수록 좋은 모형

너비 최대화

$$\max \left(\frac{2}{\|\vec{w}\|} \right) \longleftrightarrow \max \left(\frac{1}{\|\vec{w}\|} \right)$$

$$\longleftrightarrow \min(\|\vec{w}\|)$$

$$\longleftrightarrow \min \left(\frac{1}{2} \|\vec{w}\|^2 \right)$$

너비 최대화

$$\begin{aligned} \min \left(\frac{1}{2} \|\vec{w}\|^2 \right) &\iff \min \left(\frac{1}{2} \left(\sqrt{w_1^2 + w_2^2} \right)^2 \right) \\ &\iff \min \left(\frac{1}{2} (w_1^2 + w_2^2) \right) \\ &\iff \min \left(\frac{1}{2} (w_1 w_1 + w_2 w_2) \right) \\ &\iff \min \left(\frac{1}{2} (\vec{w} \cdot \vec{w}) \right) \end{aligned}$$

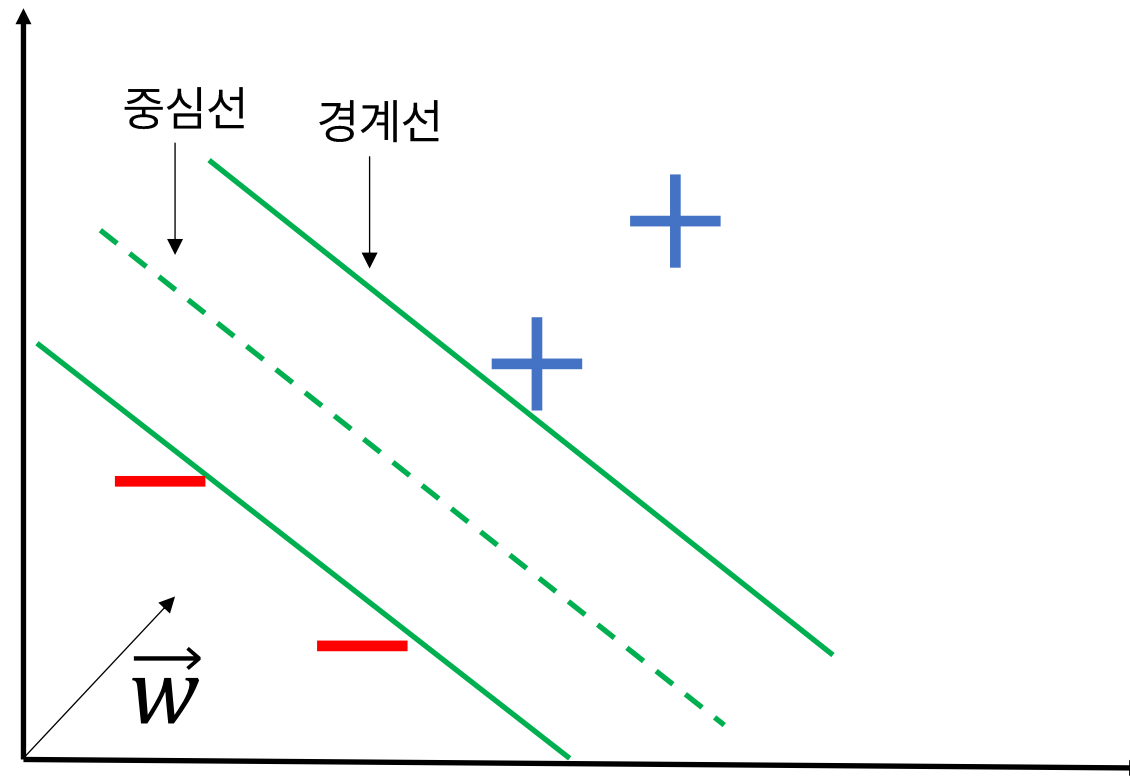
서포트 벡터 머신

너비 최대화

$$\min \left(\frac{1}{2} (\vec{w} \cdot \vec{w}) \right)$$

벡터 w 자기 자신과의 내적값의 최소화

서포트 벡터 머신



$$\vec{w} = (w_1, w_2) \quad - \text{중심선에 수직인 벡터}$$

최적화(optimization)

목적 함수
(objective function)

$$\min \left(\frac{1}{2} \|\vec{w}\|^2 \right)$$

제약 조건
(constraint)

$$y_i (\vec{w} \cdot \vec{x} + b) - 1 = 0$$

최적화(optimization)

제약 조건 $y_i(\vec{w} \cdot \vec{x} + b) - 1 = 0$ 하에서

목적 함수 $\min \left(\frac{1}{2} \|\vec{w}\|^2 \right)$ 의 최적화

목적 함수와 제약 조건을 한번에 표현하기

목적 함수
(objective function)

$$\min \left(\frac{1}{2} \|\vec{w}\|^2 \right)$$

제약 조건
(constraint)

$$y_i (\vec{w} \cdot \vec{x} + b) - 1 = 0$$

서포트 벡터 머신

$$L_p = \frac{1}{2} \|\vec{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\vec{w} \cdot \vec{x} + b) - 1]$$

서포트 벡터 머신

$$L_p = \frac{1}{2} \|\vec{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\vec{w} \cdot \vec{x} + b) - 1]$$

목적 함수
(objective function)

라그랑주 승수
(Lagrange multiplier)

제약식
(constraint)

$$L_p = \frac{1}{2} \|\vec{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\vec{w} \cdot \vec{x}_i + b) - 1]$$

$$\frac{\partial L_p}{\partial \vec{w}} = \vec{w} - \sum_{i=1}^n \alpha_i y_i \vec{x}_i = 0$$

$$\Leftrightarrow \vec{w} = \sum_{i=1}^n \alpha_i y_i \vec{x}_i$$

$$\frac{\partial L_p}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0$$

$$\Leftrightarrow \sum_{i=1}^n \alpha_i y_i = 0$$

$$\frac{\partial}{\partial \vec{w}} \left(\frac{1}{2} \|\vec{w}\|^2 \right) = \frac{\partial}{\partial \vec{w}} \left(\frac{1}{2} (\vec{w} \cdot \vec{w}) \right) = \frac{\partial}{\partial \vec{w}} \left(\frac{1}{2} (\vec{w}^T \vec{w}) \right) = \vec{w}$$

라그랑주 듀얼 함수(Lagrange Dual Function)

$$L_p = \frac{1}{2} \|\vec{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\vec{w} \cdot \vec{x}_i + b) - 1]$$

$$\vec{w} = \sum_{i=1}^n \alpha_i y_i \vec{x}_i$$

$$\sum_{i=1}^n \alpha_i y_i = 0$$

$$L_d = \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) - \sum_{i=1}^n \alpha_i [y_i \vec{w} \cdot \vec{x}_i + y_i b - 1]$$

$$\begin{aligned}
L_d &= \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) - \sum_{i=1}^n \alpha_i [y_i \vec{w} \cdot \vec{x} + y_i b - 1] \\
&= \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) - \sum_{i=1}^n (\alpha_i y_i \vec{w} \cdot \vec{x} + \alpha_i y_i b - \alpha_i) \\
&= \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) - \vec{w} \sum_{i=1}^n (\alpha_i y_i \vec{x}) + b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \\
&= \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) - \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) + b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i
\end{aligned}$$

라그랑주 듀얼 함수(Lagrange Dual Function)

$$L_d = \sum_{i=1}^n \alpha_i - \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right) \left(\sum_{i=1}^n \alpha_i y_i \vec{x}_i \right)$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j$$

데이터 자신과의 내적

최적값(optimal value)

라그랑주 프리멀 함수

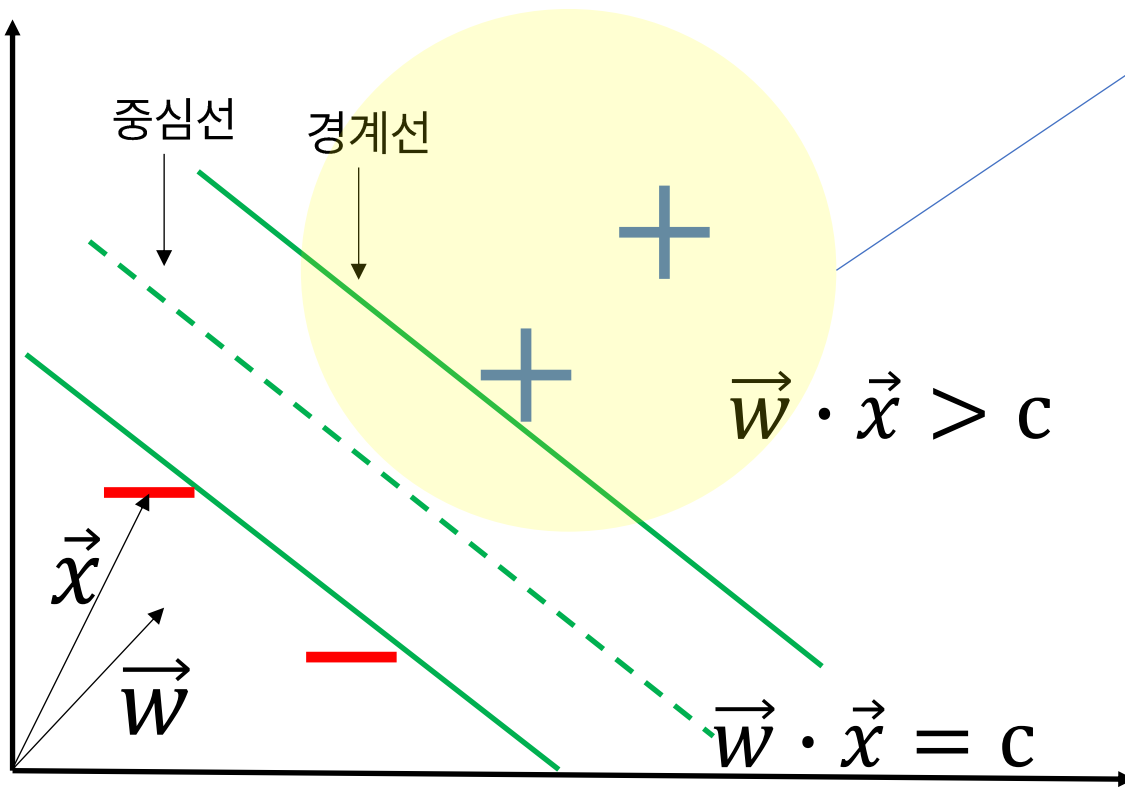
$$L_p = \frac{1}{2} \|\vec{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i (\vec{w} \cdot \vec{x}_i + b) - 1]$$

라그랑주 듀얼 함수

$$L_d = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \vec{x}_i \cdot \vec{x}_j$$

$$\vec{w} = \sum_{i=1}^n \alpha_i y_i \vec{x}_i$$

최종 결론



+ 영역에 속하려면

$$\vec{w} \cdot \vec{x} > c$$



$$\vec{w} \cdot \vec{x} + b > 0$$

Decision Rule

최종 결론

결정조건

$$\vec{w} \cdot \vec{x} + b > 0$$

최적의 w 벡터값

$$\vec{w} = \sum_{i=1}^n \alpha_i y_i \vec{x}_i$$

$$\sum_{i=1}^n \sum_{j=1}^n (\alpha_i y_i x_i x_j) + b > 0$$

자기 자신과의 내적

AI/BigData 인텐시브 과정

Section 7. 서포트 벡터 머신

Section 7-2. 서포트 벡터 머신 분류

서포트 벡터 머신 분류

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("../data/wine_data.csv")
```

```
In [2]: 1 df
```

Out[2]:

Alcohol	Malic	Ash	Alcalinity	Magesium	Phenols	Flavanoids	Nonflavanoids	Proanthocyanins	Color	Hue	Dilution	Proline	class
14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065	0
13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050	0
13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185	0
14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480	0
13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64	1.74	740	2
13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70	1.56	750	2
13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59	1.56	835	2
13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60	1.62	840	2
14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61	1.60	560	2

rows × 14 columns

서포트 벡터 머신 분류

In [3]: ▶

```
1 features = ['Alcohol', 'Malic', 'Ash', 'Alcalinity', 'Magesium', 'Phenols', 'Flavanoids',  
2           'Nonflavanoids', 'Proanthocyanins', 'Color', 'Hue', 'Dilution', 'Proline']  
3  
4 X = df[features]  
5 y = df['class']
```

서포트 벡터 머신 분류

In [4]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [5]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

서포트 벡터 머신 분류

In [6]: ▶

```
1 # 서포트 벡터 머신 학습
2 from sklearn import svm
3 clf_svm_lr = svm.SVC(kernel='linear')
4 clf_svm_lr.fit(X_tn_std, y_tn)
```

Out[6]:

```
▼ SVC
SVC(kernel='linear')
```

In [7]: ▶

```
1 # 예측
2 pred_svm = clf_svm_lr.predict(X_te_std)
3 print(pred_svm)
```

```
[0 2 1 0 1 1 0 2 1 1 2 2 0 1 2 1 0 0 1 0 1 0 0 1 1 1 1 1 1 2 0 0 1 0 0 0 2
 1 1 2 0 0 1 1 1]
```

서포트 벡터 머신 분류

Parameters:

C : float, default=1.0

Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2 penalty.

kernel : {'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callable, default='rbf'

Specifies the kernel type to be used in the algorithm. is used to pre-compute the kernel matrix from data n (n_samples, n_samples). For an intuitive visualization [boundaries with different SVM Kernels](#).

degree : int, default=3

Degree of the polynomial kernel function ('poly'). Mu

gamma : {'scale', 'auto'} or float, default='scale'

Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.

- if `gamma='scale'` (default) is passed then it uses $1 / (n_features * X.var())$ as value of gamma,
- if 'auto', uses $1 / n_features$
- if float, must be non-negative.

1.4.6. Kernel functions

The *kernel function* can be any of the following:

- linear: $\langle x, x' \rangle$.
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`.
- rbf: $\exp(-\gamma \|x - x'\|^2)$, where γ is specified by parameter `gamma`, must be greater than 0.
- sigmoid $\tanh(\gamma \langle x, x' \rangle + r)$, where r is specified by `coef0`.

서포트 벡터 머신 분류

```
In [8]: ▶ 1 # 정확도 (accuracy)
          2 from sklearn.metrics import accuracy_score
          3 accuracy = accuracy_score(y_te, pred_svm)
          4 print(accuracy)
```

1.0

```
In [9]: ▶ 1 # 정밀도 (precision)
          2 from sklearn.metrics import precision_score
          3 precision = precision_score(y_te, pred_svm, average='macro')
          4 print(precision)
```

1.0

```
In [10]: ▶ 1 # 리콜
           2 from sklearn.metrics import recall_score
           3 recall = recall_score(y_te, pred_svm, average='macro')
           4 print(recall)
```

1.0

```
In [11]: ▶ 1 # f1 스코어
           2 from sklearn.metrics import f1_score
           3 f1 = f1_score(y_te, pred_svm, average='macro')
           4 print(f1)
```

1.0

서포트 벡터 머신 분류

In [12]: ▶

```

1 # confusion matrix 확인
2 from sklearn.metrics import confusion_matrix
3 conf_matrix = confusion_matrix(y_te, pred_svm)
4 print(conf_matrix)

```

```

[[16  0  0]
 [ 0 21  0]
 [ 0  0  8]]

```

In [13]: ▶

```

1 # 분류 레포트 확인
2 from sklearn.metrics import classification_report
3 class_report = classification_report(y_te, pred_svm)
4 print(class_report)

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
1	1.00	1.00	1.00	21
2	1.00	1.00	1.00	8
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

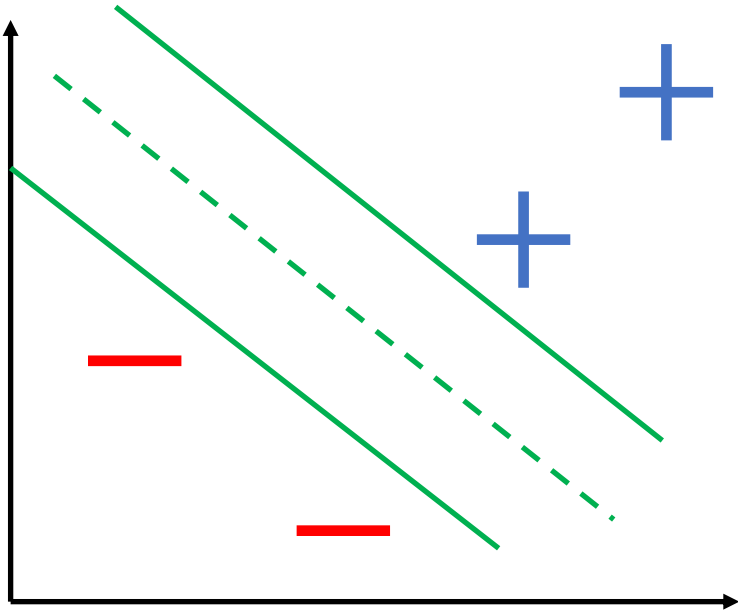
AI/BigData 인텐시브 과정

Section 7. 서포트 벡터 머신

Section 7-2. 서포트 벡터 머신 회귀

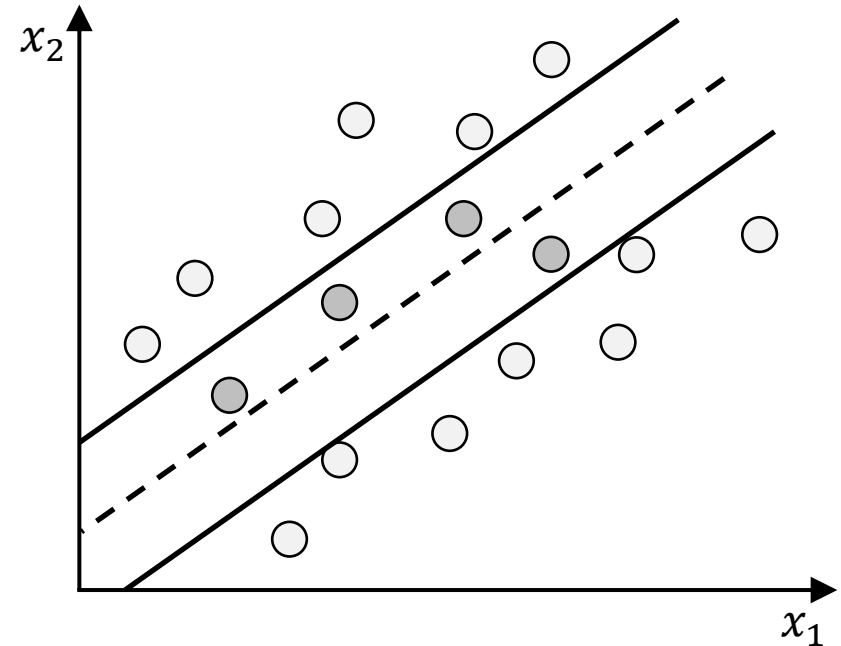
SVM classification VS SVM regressor

classification



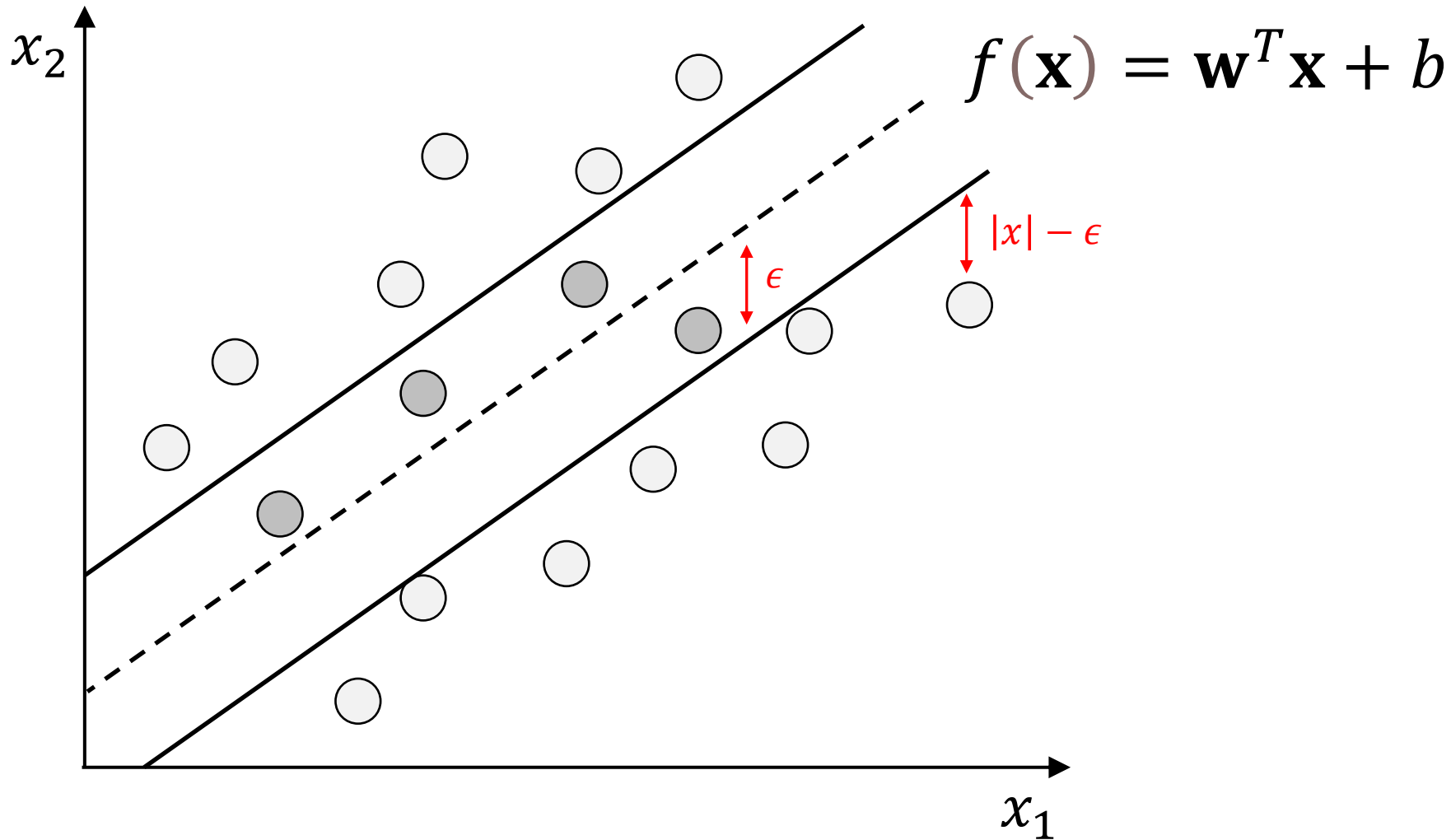
margin 내부에 데이터를 최소한으로 포함하는
초평면을 찾는 것이 목적

regressor

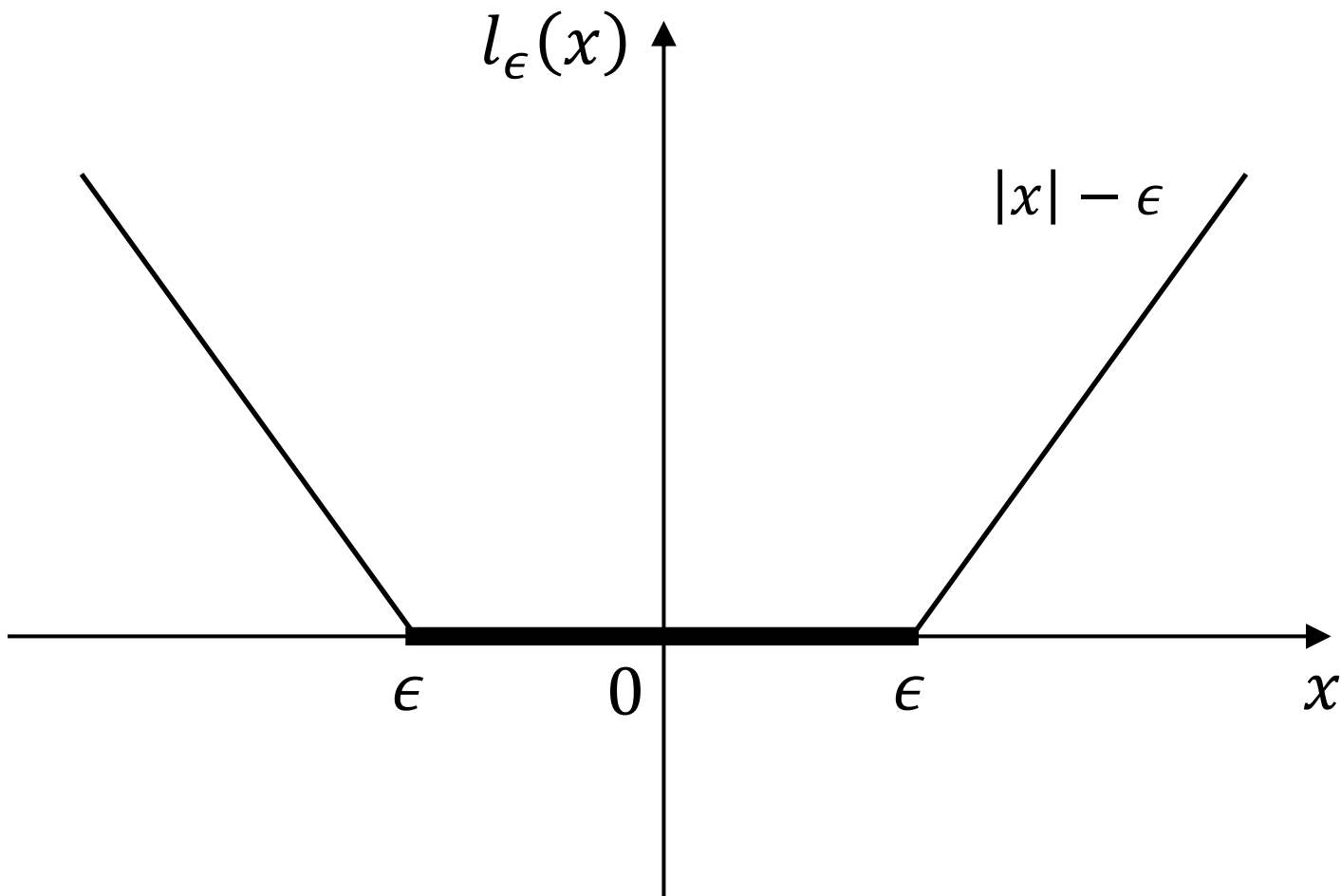


margin 내부에 데이터를 최대한으로 포함하는
초평면을 찾는 것이 목적

서포트 벡터 머신 회귀



서포트 벡터 머신 회귀



$$L_p = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n l_\epsilon(f(\mathbf{x}_i) - y_i)$$

위 식에서 $l_\epsilon(x)$ 는 ϵ -intensive 손실 이라고 하며 아래와 같이 정의 합니다. x 의 절대값이 ϵ 내부에 존재하면 0이고, ϵ 외부에 존재한다면 $|x| - \epsilon$ 값을 가집니다.

$$l_\epsilon(x) = \begin{cases} 0, & |x| \leq \epsilon \\ |x| - \epsilon, & \text{otherwise} \end{cases}$$

서포트 벡터 머신 회귀

$$\min_{\mathbf{w}, b, \xi, \psi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n l_{\epsilon}(f(\mathbf{x}_i) - y_i)$$
$$\text{subject to } \begin{cases} f(\mathbf{x}_i) - y_i \leq \epsilon + \xi_i \\ y_i - f(\mathbf{x}_i) \leq \epsilon + \psi_i \\ \xi_i \geq 0, \psi_i \geq 0 \end{cases}$$

서포트 벡터 머신 회귀

$$\begin{aligned} L_p = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \psi_i) - \sum_{i=1}^n \gamma_i \xi_i - \sum_{i=1}^n \delta_i \psi_i \\ & + \sum_{i=1}^n \alpha_i (f(x_i) - y_i - \epsilon - \xi_i) + \sum_{i=1}^n \beta_i (y_i - f(x_i) - \epsilon - \psi_i) \end{aligned}$$

서포트 벡터 머신 회귀

```
In [1]: 1 import pandas as pd
        2 df = pd.read_csv("./data/house_prices.csv")
```

```
In [2]: 1 df
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.4
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.6
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.9
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.0
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.9

506 rows × 14 columns

서포트 벡터 머신 회귀

```
In [3]: 1 df['MEDV'].unique()
```

```
Out[3]: array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9, 27.1, 16.5, 18.9, 15. ,
               21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 13.6, 19.6, 15.2, 14.5,
               15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 13.2, 13.1, 13.5, 20. ,
               24.7, 30.8, 34.9, 26.6, 25.3, 21.2, 19.3, 14.4, 19.4, 19.7, 20.5,
               25. , 23.4, 35.4, 31.6, 23.3, 18.7, 16. , 22.2, 33. , 23.5, 22. ,
               17.4, 20.9, 24.2, 22.8, 24.1, 21.4, 20.8, 20.3, 28. , 23.9, 24.8,
               22.5, 23.6, 22.6, 20.6, 28.4, 38.7, 43.8, 33.2, 27.5, 26.5, 18.6,
               20.1, 19.5, 19.8, 18.8, 18.5, 18.3, 19.2, 17.3, 15.7, 16.2, 18. ,
               14.3, 23. , 18.1, 17.1, 13.3, 17.8, 14. , 13.4, 11.8, 13.8, 14.6,
               15.4, 21.5, 15.3, 17. , 41.3, 24.3, 27. , 50. , 22.7, 23.8, 22.3,
               19.1, 29.4, 23.2, 24.6, 29.9, 37.2, 39.8, 37.9, 32.5, 26.4, 29.6,
               32. , 29.8, 37. , 30.5, 36.4, 31.1, 29.1, 33.3, 30.3, 34.6, 32.9,
               42.3, 48.5, 24.4, 22.4, 28.1, 23.7, 26.7, 30.1, 44.8, 37.6, 46.7,
               31.5, 31.7, 41.7, 48.3, 29. , 25.1, 17.6, 24.5, 26.2, 42.8, 21.9,
               44. , 36. , 33.8, 43.1, 48.8, 31. , 36.5, 30.7, 43.5, 20.7, 21.1,
               25.2, 35.2, 32.4, 33.1, 35.1, 45.4, 46. , 32.2, 28.5, 37.3, 27.9,
               28.6, 36.1, 28.2, 16.1, 22.1, 19. , 32.7, 31.2, 17.2, 16.8, 10.2,
               10.4, 10.9, 11.3, 12.3,  8.8,  7.2, 10.5,  7.4, 11.5, 15.1,  9.7,
               12.5,  8.5,  5. ,  6.3,  5.6, 12.1,  8.3, 11.9, 17.9, 16.3,  7. ,
               7.5,  8.4, 16.7, 14.2, 11.7, 11. ,  9.5, 14.1,  9.6,  8.7, 12.8,
               10.8, 14.9, 12.6, 13. , 16.4, 17.7, 12. , 21.8,  8.1])
```

Section

서포트 벡터 머신 회귀

In [4]: ▶

1 df.columns

Out[4]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT'], dtype='object')

In [5]: ▶

1 features = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT']
2
3 X = df[features]
4 y = df['MEDV']

In [6]: ▶

1 X

Out[6]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows x 13 columns

서포트 벡터 머신 회귀

In [7]: ▶

```
1 # 트레이닝/테스트 데이터 분할
2 from sklearn.model_selection import train_test_split
3 X_tn, X_te, y_tn, y_te=train_test_split(X,y,random_state=0)
```

In [8]: ▶

```
1 #데이터 표준화
2 from sklearn.preprocessing import StandardScaler
3 std_scale = StandardScaler()
4 std_scale.fit(X_tn)
5 X_tn_std = std_scale.transform(X_tn)
6 X_te_std = std_scale.transform(X_te)
```

서포트 벡터 머신 회귀

```
In [9]: ▶ 1 # 서포트 벡터 머신 학습
          2 from sklearn.svm import SVR
          3 clf_svm_lr = SVR(kernel='linear')
          4 clf_svm_lr.fit(X_tn_std, y_tn)
```

Out[9]:

SVR

SVR(kernel='linear')

```
In [10]: ▶ 1 # 예측
          2 svm_pred = clf_svm_lr.predict(X_te_std)
          3 print(svm_pred)
```

```
[24.52947544 20.35423732 26.47676448  9.74009208 21.0378301  18.85874
18.6263904  20.65306829 16.3571043  18.44120108  1.83859761 14.77921945
16.8165412   3.39294294 37.49176197 32.0579395  20.41257183 35.48866506
29.23822418 21.81376385 24.24218256 21.49031908 19.78380253 28.84780636
20.97310945  2.94925643 17.38496471 17.35019198 35.64837325 19.77476059
16.76254231 17.03973933 19.33992199 22.38892841 26.47086708 17.24991582
10.96895118 20.26465974 15.48478266 14.59886084 25.04138918 20.30869651]
```

```
In [11]: ▶ 1 # 모형 평가-MSE
          2 from sklearn.metrics import mean_squared_error
          3 mse = mean_squared_error(y_te, svm_pred)
          4 print(mse)
```

```
35.24178412488106
```

감사합니다.

Q & A